PROJECT DOCUMENTATION PORTFOLIO TEMPLATE

Standard Operating Blueprint for Engineering & Data Science Repositories

> **Note for Authors:** Use this editable template as a foundation for documenting your software projects. Clean repository structure and business-focused READMEs significantly increase code review approval rates.

1. Pre-Publishing Quality Checklist

- ☐ **Modular Codebase:** Business logic partitioned into `/src` and decoupled from entry points.
- ☐ **Environment Isolation:** Explicit dependency locks via `requirements.txt`, `package.json`, or `Dockerfile`.
- ☐ **Automated Verification:** Unit tests placed in `/tests` directory with passing execution status.
- ☐ **Executive README:** Formatted summary detailing high-level architecture and quantifiable results.

2. Standard Repository File Hierarchy

```
my-portfolio-project/
├── README.md               # Executive summary & setup documentation
├── LICENSE                 # Open-source distribution license
├── .gitignore              # Dependency & credential exclusions
├── requirements.txt        # Locked environment dependencies
├── Dockerfile              # Container deployment specification
├── docs/                   # Visual architecture & API specifications
│   ├── architecture.png
│   └── API_SPEC.md
├── src/                    # Modular application code
│   ├── __init__.py
│   ├── data_pipeline.py    # Ingestion & transformation code
│   └── service.py          # Core logic & API endpoints
└── tests/                  # Automated verification suite
    └── test_service.py
```

3. Technology Stack Reference Matrix

Layer	Recommended Stack	Architectural Rationale
Backend / API	FastAPI / Node.js (TS)	Asynchronous I/O performance with automatic schema validation.
Data Store	PostgreSQL / Redis	ACID compliance for transactional state; low-latency cache layer.
Runtime / Ops	Docker / GitHub Actions	Containerized environment parity and automated CI test runs.

4. Production README Template

```
# [Project Title]

> **Tagline:** One-sentence explanation of what the application does and who it
serves.

## Executive Summary
* **Problem:** Real-world challenge or bottleneck addressed by this project.
* **Solution:** Architectural approach and key technical components built.
* **Impact:** Quantifiable metrics (e.g., "Reduced query latency by 45% under 10k
RPS").

## Installation & Local Execution
1. Clone repository: git clone https://github.com/user/project.git
2. Setup environment: python -m venv venv && source venv/bin/activate
3. Install dependencies: pip install -r requirements.txt
4. Execute application: python src/service.py

## Key Trade-offs & Engineering Decisions
* **Decision 1:** Reason for choosing relational SQL over document stores.
* **Decision 2:** Asynchronous execution model selection to prevent API blocking.
```